

Practicle-2(A)

Aim: To create and implement a Java Thermostat class using encapsulation with private fields, constructors and constructor chaining, static and final variables, temperature validation with minimum and maximum limits, getter methods, and loops to raise and lower temperature while tracking the number of active thermostat objects.

Code Screenshot:

```
public class Thermostat {  
  
    private String location;  
    private int temperature;  
    private static final int MIN = 16;  
    private static final int MAX = 30;  
    private static int activeCount = 0;  
  
    public Thermostat(String location, int startTemp) {  
        this.location = location;  
  
        if (startTemp >= MIN && startTemp <= MAX) {  
            this.temperature = startTemp;  
        } else {  
            this.temperature = 22;  
        }  
        activeCount++;  
    }  
  
    public void raise(){  
        if (temperature < MAX) {  
            temperature++;  
        } else {  
            System.out.println("Already at maximum (30)");  
        }  
    }  
  
    public void lower() {  
        if (temperature > MIN) {  
            temperature--;  
        } else {  
            System.out.println("Already at minimum (16)");  
        }  
    }  
  
    public int getTemperature() {  
        return temperature;  
    }  
}
```

```

public static int getActiveCount() {
    return activeCount;
}
Run | Debug | Run main | Debug main
public static void main(String[] args){
    Thermostat th1 = new Thermostat(location: "classRoom", startTemp: 22);
    System.out.println("Temperature: " + th1.getTemperature());

    System.out.println(x: "Raising Temperature:");
    for (int i = 0; i < 10; i++) {
        th1.raise();
        System.out.println("Temperature: " + th1.getTemperature());
    }
    System.out.println(x: "\nLowering Temperature:");
    for (int i = 0; i < 20; i++) {
        th1.lower();
        System.out.println("Temperature: " + th1.getTemperature());
    }
}
}

```

Output Screenshot:

```

PS C:\Users\Aryan\Documents\Charusat\Sem-3\Java\Practicle-2> java Thermostat
Temperature: 22
Raising Temperature:
Temperature: 23
Temperature: 24
Temperature: 25
Temperature: 26
Temperature: 27
Temperature: 28
Temperature: 29
Temperature: 30
Already at maximum (30)
Temperature: 30
Already at maximum (30)
Temperature: 30

Lowering Temperature:
Temperature: 29
Temperature: 28
Temperature: 27
Temperature: 26
Temperature: 25
Temperature: 24
Temperature: 23
Temperature: 22
Temperature: 21
Temperature: 20
Temperature: 19
Temperature: 18
Temperature: 17
Temperature: 16
Already at minimum (16)
Temperature: 16
Already at minimum (16)
Temperature: 16
Already at minimum (16)
Temperature: 16
Already at minimum (16)
Temperature: 16
Already at minimum (16)
Temperature: 16
Already at minimum (16)
Temperature: 16

```

Practicle-2(B)

Aim: To create and implement a Java CinemaShow class using encapsulation with private and final fields, constructors and constructor chaining, static variables, booking and cancellation methods with capacity validation, getter methods, and a main program to perform booking and cancellation operations while tracking total booked seats.

Code Screenshot:

```
class CinemaShow {
    private String title;
    private int seatsAvailable;
    private final int capacity;
    private static int totalBooked = 0;

    CinemaShow(String title, int capacity) {
        this.title = title;
        this.capacity = capacity;
        this.seatsAvailable = capacity;
    }

    CinemaShow(String title) {
        this(title, capacity: 100);
    }

    boolean book(int n) {
        if (n <= seatsAvailable) {
            seatsAvailable -= n;
            totalBooked += n;
            return true;
        }
        return false;
    }

    void cancel(int n) {
        int bookedSeats = capacity - seatsAvailable;

        if (n <= bookedSeats) {
            seatsAvailable += n;
            totalBooked -= n;
        }
    }

    int getSeatsAvailable() {
        return seatsAvailable;
    }

    static int getTotalBooked() {
        return totalBooked;
    }
}
```

```

public class CinemaTest {
    Run | Debug | Run main | Debug main
    public static void main(String[] args) {

        CinemaShow userA = new CinemaShow(title: "Show A", capacity: 5);

        if (userA.book(n: 3))
            System.out.println("User A) -> booked 3 | remaining "
                               + userA.getSeatsAvailable());

        if (userA.book(n: 1))
            System.out.println("User A) -> booked 1 | remaining "
                               + userA.getSeatsAvailable());

        boolean result = userA.book(n: 3);
        System.out.println("User A) -> booked 3 -> " + result
                           + " | remaining " + userA.getSeatsAvailable());

        userA.cancel(n: 1);
        System.out.println("User A) -> cancel 1 | remaining "
                           + userA.getSeatsAvailable());

        CinemaShow userB = new CinemaShow(title: "Show B");

        if (userB.book(n: 5))
            System.out.println("User B) -> booked 5 | remaining "
                               + userB.getSeatsAvailable());

        System.out.println();
        System.out.println("Total Booking -> "
                           + CinemaShow.getTotalBooked());
    }
}

```

Output Screenshot:

```

PS C:\Users\Aryan\Documents\Charusat\Sem-3\Java\Practicle-2> java CinemaTest
User A) -> booked 3 | remaining 2
User A) -> booked 1 | remaining 1
User A) -> booked 3 -> false | remaining 1
User A) -> cancel 1 | remaining 2
User B) -> booked 5 | remaining 95

Total Booking -> 8

```

- **Question-Answer: -**

1. **What is encapsulation, and how do private fields with public getters enforce it?**

- Encapsulation: It means keeping data private and accessing it through public methods like getters.

2. **Why is accountNumber declared final?**

- accountNumber is final: It cannot be changed after it is assigned.

3. **What is the difference between a static field and an instance field when generating ids?**

- **Static vs instance field:** A static field is shared by all objects, while an instance field is separate for each object.

Learning Outcome:

The learning outcome is to understand and implement the concepts of classes, objects, and constructors in Java. Students will learn constructor overloading and constructor chaining to create objects efficiently. They will understand encapsulation by using private data members and public methods. They will learn how static and final variables are used to maintain shared and constant values. Students will practice applying conditional statements, loops, and boundary checking in real-world problems. They will also improve their ability to design simple Java programs using object-oriented programming concepts.

GitHub Repositories:-

<https://github.com/d26it116/OOP/tree/159f27c0b1a369ac4529973d98446af6a77befb1/Practicle-2>